	COCO256 × 256	OpenImages 256 × 256	OpenImages 512 × 512
Method	FID↓	FID↓	FID↓
LostGAN-V2 [87]	42.55	-	-
OC-GAN [89]	41.65	-	-
SPADE [62]	41.11	-	-
VQGAN+T [37]	56.58	45.33	48.11
LDM-8 (100 steps, ours)	42.06†	-	-
LDM-4 (200 steps, ours)	**40.91***	**32.02**	**35.80**

Table 9. Quantitative comparison of our layout-to-image models on the COCO [4] and OpenImages [49] datasets. †: Training from scratch on COCO; *: Finetuning from OpenImages.

Method	FID↓	IS↑	Precision↑	Recall↑	N_{params}	
SR3 [72]	11.30	-	-	-	625M	-
ImageBART [21]	21.19	-	-	-	3.5B	-
ImageBART [21]	7.44	-	-	-	3.5B	0.05 acc. rate*
VQGAN+T [23]	17.04	$70.6_{\pm 1.8}$	-	-	1.3B	-
VQGAN+T [23]	5.88	$\mathbf{304.8}_{\pm 3.6}$	-	-	1.3B	0.05 acc. rate*
BigGan-deep [3]	6.95	$203.6_{\pm 2.6}$	**0.87**	0.28	340M	-
ADM [15]	10.94	100.98	0.69	**0.63**	554M	250 DDIM steps
ADM-G [15]	4.59	186.7	0.82	0.52	608M	250 DDIM steps
ADM-G,ADM-U [15]	3.85	221.72	0.84	0.53	n/a	2 × 250 DDIM steps
CDM [31]	4.88	$158.71_{\pm 2.26}$	-	-	n/a	2 × 100 DDIM steps
LDM-8 (ours)	17.41	$72.92_{\pm 2.6}$	0.65	0.62	395M	200 DDIM steps, 2.9M train steps, batch size 64
LDM-8-G (ours)	8.11	$190.43_{\pm 2.60}$	0.83	0.36	506M	200 DDIM steps, classifier scale 10, 2.9M train steps, batch size 64
LDM-8 (ours)	15.51	$79.03_{\pm 1.03}$	0.65	**0.63**	395M	200 DDIM steps, 4.8M train steps, batch size 64
LDM-8-G (ours)	7.76	$209.52_{\pm 4.24}$	0.84	0.35	506M	200 DDIM steps, classifier scale 10, 4.8M train steps, batch size 64
LDM-4 (ours)	10.56	$103.49_{\pm 1.24}$	0.71	0.62	400M	250 DDIM steps, 178K train steps, batch size 1200
LDM-4-G (ours)	3.95	$178.22_{\pm 2.43}$	0.81	0.55	400M	250 DDIM steps, unconditional guidance [32] scale 1.25, 178K train steps, batch size 1200
LDM-4-G (ours)	**3.60**	$247.67_{\pm 5.59}$	**0.87**	0.48	400M	250 DDIM steps, unconditional guidance [32] scale 1.5, 178K train steps, batch size 1200

Table 10. Comparison of a class-conditional ImageNet *LDM* with recent state-of-the-art methods for class-conditional image generation on the ImageNet [12] dataset.*: Classifier rejection sampling with the given rejection rate as proposed in [67].

D.5. Sample Quality vs. V100 Days (Continued from Sec. 4.1)

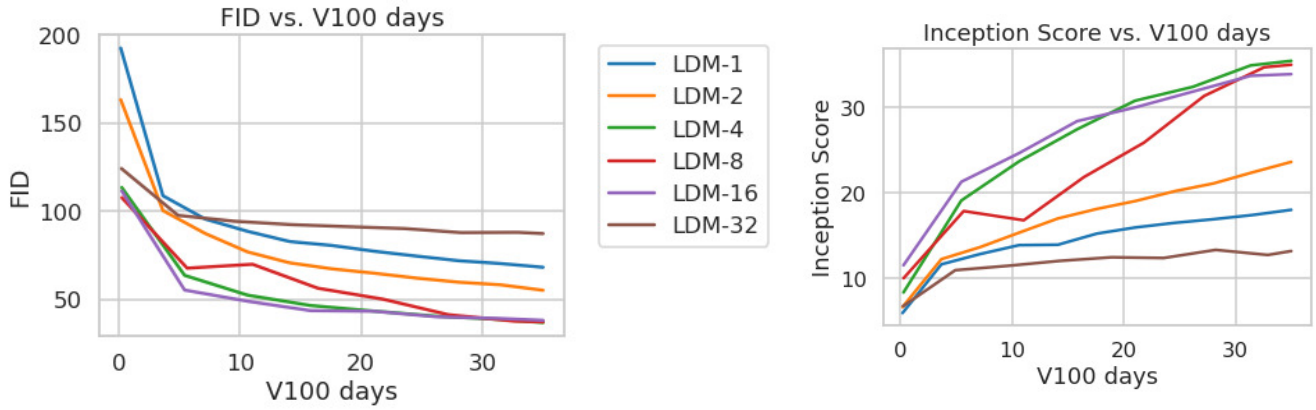

Figure 17. For completeness we also report the training progress of class-conditional *LDMs* on the ImageNet dataset for a fixed number of 35 V100 days. Results obtained with 100 DDIM steps [84] and $\kappa = 0$. FIDs computed on 5000 samples for efficiency reasons.

For the assessment of sample quality over the training progress in Sec. 4.1, we reported FID and IS scores as a function of train steps. Another possibility is to report these metrics over the used resources in V100 days. Such an analysis is additionally provided in Fig. 17, showing qualitatively similar results.

Method	FID ↓	IS ↑	PSNR ↑	SSIM ↑
Image Regression [72]	15.2	121.1	**27.9**	**0.801**
SR3 [72]	5.2	**180.1**	26.4	0.762
LDM-4 (ours, 100 steps)	**2.8**†/**4.8**‡	166.3	$24.4_{\pm 3.8}$	$0.69_{\pm 0.14}$
LDM-4 (ours, 50 steps, guiding)	4.4†/6.4‡	153.7	$25.8_{\pm 3.7}$	$0.74_{\pm 0.12}$
LDM-4 (ours, 100 steps, guiding)	4.4†/6.4‡	154.1	$25.7_{\pm 3.7}$	$0.73_{\pm 0.12}$
LDM-4 (ours, 100 steps, +15 ep.)	**2.6**† / **4.6**‡	$169.76_{\pm 5.03}$	$24.4_{\pm 3.8}$	$0.69_{\pm 0.14}$
Pixel-DM (100 steps, +15 ep.)	5.1† / 7.1‡	$163.06_{\pm 4.67}$	$24.1_{\pm 3.3}$	$0.59_{\pm 0.12}$

Table 11. $\times 4$ upscaling results on ImageNet-Val. (256^2); †: FID features computed on validation split, ‡: FID features computed on train split. We also include a pixel-space baseline that receives the same amount of compute as *LDM-4*. The last two rows received 15 epochs of additional training compared to the former results.

D.6. Super-Resolution

For better comparability between LDMs and diffusion models in pixel space, we extend our analysis from Tab. 5 by comparing a diffusion model trained for the same number of steps and with a comparable number [1] of parameters to our LDM. The results of this comparison are shown in the last two rows of Tab. 11 and demonstrate that LDM achieves better performance while allowing for significantly faster sampling. A qualitative comparison is given in Fig. 20 which shows random samples from both LDM and the diffusion model in pixel space.

D.6.1 LDM-BSR: General Purpose SR Model via Diverse Image Degradation

Figure 18. *LDM-BSR* generalizes to arbitrary inputs and can be used as a general-purpose upsampler, upscaling samples from a class-conditional *LDM* (image *cf*. Fig. 4) to 1024^2 resolution. In contrast, using a fixed degradation process (see Sec. 4.4) hinders generalization.

To evaluate generalization of our LDM-SR, we apply it both on synthetic LDM samples from a class-conditional ImageNet model (Sec. 4.1) and images crawled from the internet. Interestingly, we observe that LDM-SR, trained only with a bicubicly downsampled conditioning as in [72], does not generalize well to images which do not follow this pre-processing. Hence, to obtain a superresolution model for a wide range of real world images, which can contain complex superpositions of camera noise, compression artifacts, blurr and interpolations, we replace the bicubic downsampling operation in LDM-SR with the degration pipeline from [105]. The BSR-degradation process is a degradation pipline which applies JPEG compressions noise, camera sensor noise, different image interpolations for downsampling, Gaussian blur kernels and Gaussian noise in a random order to an image. We found that using the bsr-degredation process with the original parameters as in [105] leads to a very strong degradation process. Since a more moderate degradation process seemed apppropiate for our application, we adapted the parameters of the bsr-degradation (our adapted degradation process can be found in our code base at `https://github.com/CompVis/latent-diffusion`). Fig. 18 illustrates the effectiveness of this approach by directly comparing *LDM-SR* with *LDM-BSR*. The latter produces images much sharper than the models confined to a fixed pre-processing, making it suitable for real-world applications. Further results of *LDM-BSR* are shown on LSUN-cows in Fig. 19.

[1] It is not possible to exactly match both architectures since the diffusion model operates in the pixel space

E. Implementation Details and Hyperparameters

E.1. Hyperparameters

We provide an overview of the hyperparameters of all trained *LDM* models in Tab. 12, Tab. 13, Tab. 14 and Tab. 15.

	CelebA-HQ 256×256	FFHQ 256×256	LSUN-Churches 256×256	LSUN-Bedrooms 256×256
f	4	4	8	4
z-shape	$64 \times 64 \times 3$	$64 \times 64 \times 3$	-	$64 \times 64 \times 3$
$\lvert\mathcal{Z}\rvert$	8192	8192	-	8192
Diffusion steps	1000	1000	1000	1000
Noise Schedule	linear	linear	linear	linear
N_{params}	274M	274M	294M	274M
Channels	224	224	192	224
Depth	2	2	2	2
Channel Multiplier	1,2,3,4	1,2,3,4	1,2,2,4,4	1,2,3,4
Attention resolutions	32, 16, 8	32, 16, 8	32, 16, 8, 4	32, 16, 8
Head Channels	32	32	24	32
Batch Size	48	42	96	48
Iterations*	410k	635k	500k	1.9M
Learning Rate	9.6e-5	8.4e-5	5.e-5	9.6e-5

Table 12. Hyperparameters for the unconditional *LDMs* producing the numbers shown in Tab. 1. All models trained on a single NVIDIA A100.

	LDM-1	*LDM-2*	*LDM-4*	*LDM-8*	*LDM-16*	*LDM-32*
z-shape	$256 \times 256 \times 3$	$128 \times 128 \times 2$	$64 \times 64 \times 3$	$32 \times 32 \times 4$	$16 \times 16 \times 8$	$88 \times 8 \times 32$
$\lvert\mathcal{Z}\rvert$	-	2048	8192	16384	16384	16384
Diffusion steps	1000	1000	1000	1000	1000	1000
Noise Schedule	linear	linear	linear	linear	linear	linear
Model Size	396M	391M	391M	395M	395M	395M
Channels	192	192	192	256	256	256
Depth	2	2	2	2	2	2
Channel Multiplier	1,1,2,2,4,4	1,2,2,4,4	1,2,3,5	1,2,4	1,2,4	1,2,4
Number of Heads	1	1	1	1	1	1
Batch Size	7	9	40	64	112	112
Iterations	2M	2M	2M	2M	2M	2M
Learning Rate	4.9e-5	6.3e-5	8e-5	6.4e-5	4.5e-5	4.5e-5
Conditioning	CA	CA	CA	CA	CA	CA
CA-resolutions	32, 16, 8	32, 16, 8	32, 16, 8	32, 16, 8	16, 8, 4	8, 4, 2
Embedding Dimension	512	512	512	512	512	512
Transformers Depth	1	1	1	1	1	1

Table 13. Hyperparameters for the conditional *LDMs* trained on the ImageNet dataset for the analysis in Sec. 4.1. All models trained on a single NVIDIA A100.

E.2. Implementation Details

E.2.1 Implementations of τ_θ for conditional *LDMs*

For the experiments on text-to-image and layout-to-image (Sec. 4.3.1) synthesis, we implement the conditioner τ_θ as an unmasked transformer which processes a tokenized version of the input y and produces an output $\zeta := \tau_\theta(y)$, where $\zeta \in \mathbb{R}^{M \times d_\tau}$. More specifically, the transformer is implemented from N transformer blocks consisting of global self-attention layers, layer-normalization and position-wise MLPs as follows[2]:

[2]adapted from https://github.com/lucidrains/x-transformers